

数字逻辑实验报告

实验 4

学 期	2024-2025 学年第 1 学期	实验日期	2024/11/26	
学 院	计算机学院	组 号	学 号	姓 名
专 业	计算机科学与技术（实验班）	15	23071005	侯景祺
班 级	230710			
评 阅 内 容				
任务一	任务二	总结		成 绩
题 目	实验 4：状态机电路设计			
一、实验目的 通过本实验掌握典型状态机电路的功能和特点;掌握摩尔型和米利型状态机的基本分析方法和设计方法;掌握使用硬件描述语言设计状态机电路的方法;巩固和加深对课程基本理论知识的理解。 通过交通灯、流水灯、序列检测器等电路的设计与测试，掌握状态机电路的分析方法和设计方法;学会使用 Verilog HDL 设计状态机电路 二、任务一设计与实现 1. 要求 输出用 LED 显示，显示模式为 LED 灯从左至右或从右至左轮流点亮，也可自定义显示模式。根据实现模式，画出状态图。 用 Verilog 编写状态机程序。 2. 设计思路 分频器只需分离 1hz 的信号，因此可以通过简化分频器代码，令其只输出 1hz 信号。将流水灯设置为初始状态全灭，然后由右向左流水。 <pre>graph LR; Input[输入] --> Divider[分频器]; Divider -- "clk-1Hz" --> WaterfallLight[流水灯];</pre>				

3. 详细设计

分频器为实验五中分频器的改造版本。如下图所示。

```

1  module frequency_divider(clk_50mhz,rst,clk_1hz);
2
3  input  clk_50mhz,rst;
4  output clk_1hz;
5
6  reg clk_1hz;
7  reg [31:0]cnt1;
8  //parameter A=50000000;
9  parameter A = 2;
10
11 always@(posedge clk_50mhz)
12     begin
13         if(!rst)
14             begin
15                 cnt1<=1'b0;
16                 clk_1hz<=1'b0;
17             end
18         else
19             if(cnt1<A/2-1)
20                 cnt1<=cnt1+1'b1;
21             else
22                 begin
23                     cnt1<=1'b0;
24                     clk_1hz<=~clk_1hz;
25                 end
26             end
27     endmodule
    
```

流水灯电路如下。定义好了 17 种状态（ste）的值，可以在时序电路部分转换状态。

```

1  module liu_water_light(reset,out,clk_lhz);
2  input reset;
3  input clk_lhz;
4
5  output reg[15:0]out;
6  reg [4:0]state;
7
8  parameter ste0=0,ste1=1,ste2=2,ste3=3,ste4=4,ste5=5,ste6=6,ste7=7,ste8=8,ste9=9,ste10=10,ste11=11,ste12=12,ste13=13,ste14=14,ste15=15,ste16=16;
9
10 always @(state) begin
11     case (state)
12         ste0:out=16'b1111_1111_1111_1111;
13         ste1:out=16'b1111_1111_1111_1110;
14         ste2:out=16'b1111_1111_1111_1101;
15         ste3:out=16'b1111_1111_1111_1011;
16         ste4:out=16'b1111_1111_1111_0111;
17         ste5:out=16'b1111_1111_1110_1111;
18         ste6:out=16'b1111_1111_1101_1111;
19         ste7:out=16'b1111_1111_1011_1111;
20         ste8:out=16'b1111_1111_0111_1111;
21         ste9:out=16'b1111_1110_1111_1111;
22         ste10:out=16'b1111_1101_1111_1111;
23         ste11:out=16'b1111_1011_1111_1111;
24         ste12:out=16'b1111_0111_1111_1111;
25         ste13:out=16'b1110_1111_1111_1111;
26         ste14:out=16'b1101_1111_1111_1111;
27         ste15:out=16'b1011_1111_1111_1111;
28         ste16:out=16'b0111_1111_1111_1111;
29         default: out=16'b1111_1111_1111_1111;
30     endcase
31 end
32
33 always @(posedge clk_lhz)
34 begin
35     if(!reset)
36         state=ste0;
37     else
38         case (state)
39             ste0:state<=ste1;
40             ste1:state<=ste2;
41             ste2:state<=ste3;
42             ste3:state<=ste4;
43             ste4:state<=ste5;
44             ste5:state<=ste6;
45             ste6:state<=ste7;
46             ste7:state<=ste8;
47             ste8:state<=ste9;
48             ste9:state<=ste10;
49             ste10:state<=ste11;
50             ste11:state<=ste12;
51             ste12:state<=ste13;
52             ste13:state<=ste14;
53             ste14:state<=ste15;
54             ste15:state<=ste16;
55             ste16:state<=ste0;
56             default: state<=ste0;
57         endcase
58     end
59 endmodule

```

如下，为顶层文件，它调度了分频器、流水灯电路。

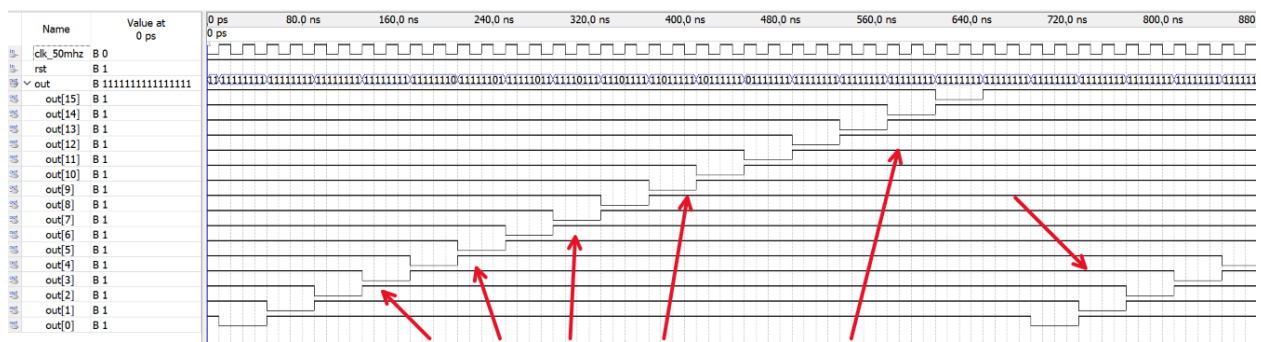
```

1  module top(clk_50mhz,rst,out);
2
3  input clk_50mhz, rst;
4
5  wire clk_lhz;
6  output [15:0] out;
7
8  frequency_divider(.clk_50mhz(clk_50mhz), .rst(rst), .clk_lhz(clk_lhz));
9
10 liu_water_light(.reset(rst), .out(out) ,.clk_lhz(clk_lhz)); //set instance liu_water_light
11
12 endmodule

```

4. 仿真实验

如下图，clk_50mhz 按照时序波动时，可以期待流水灯的持续变化。



如红色箭头所示，输出信号呈现“流水状”逐个出现
可期待灯逐个亮起，并在16个灯亮后循环亮起

5. 引脚分配

如图，clk 接入内置信号输出；out 输出接到 LED 灯输出；rst 为暂停信号，接到开关。

Node Name	Direction	Location	I/O Bank
clk_50mhz	Input	PIN_T1	2
out[15]	Output	PIN_U12	4
out[14]	Output	PIN_V12	4
out[13]	Output	PIN_V15	4
out[12]	Output	PIN_W13	4
out[11]	Output	PIN_W15	4
out[10]	Output	PIN_Y17	4
out[9]	Output	PIN_R16	4
out[8]	Output	PIN_T17	5
out[7]	Output	PIN_E11	7
out[6]	Output	PIN_C13	7
out[5]	Output	PIN_F11	7
out[4]	Output	PIN_C15	7
out[3]	Output	PIN_E14	7
out[2]	Output	PIN_B7	8
out[1]	Output	PIN_B8	8
out[0]	Output	PIN_B9	8
rst	Input	PIN_N18	5

6. 实验现象

工作台上的 LED 灯从左到右间隔相同的时间依次闪烁。在拨动 rst 开关后，程序阻塞。

三、任务二设计与实现

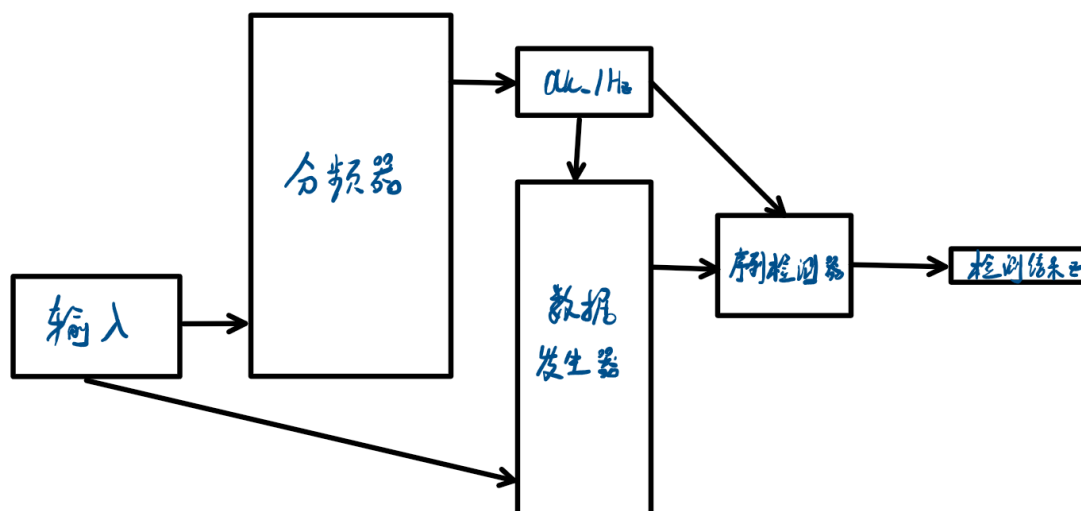
1. 要求

设计一个学号序列检测器，序列为同组两位同学学号末两位相加，如相加后低于 16，需在和的基础上加 30(检测序列为结果对应的 8421BCD 码，如结果为 34，检测序列为 00110100)。

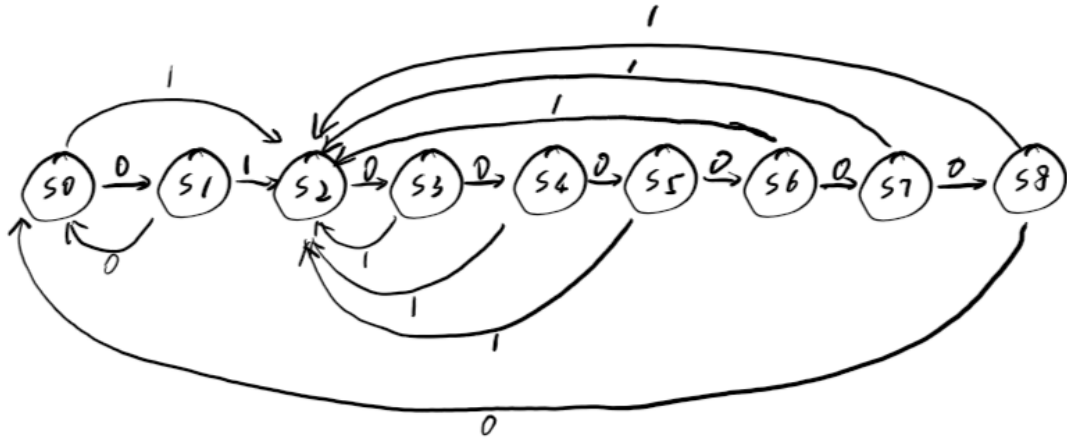
采用 Verilog 实现状态机。

2. 设计思路

此实验分频器与任务一类似，将时钟信号输入到数据发生器，产生单个编码并传入至检测器，从而检测序列。检测器为 Moore 型。设计图如下。



可以使用的循环逻辑如下。



3. 详细设计

对于序列检测器，序列为同组两位同学学号末两位相加，如相加后低于 16，需在和的基础上加 30。
对于我而言，为 40。

```

1  module seqdetector(clk, x, reset, z, c_state, n_state);
2
3      input clk, x, reset; // 时钟信号、输入检测位、复位信号
4      output reg z; // 输出信号，表示检测成功
5      output reg [3:0] c_state, n_state; // 当前状态和下一状态
6
7      // 状态定义
8      parameter S0 = 0, S1 = 1, S2 = 2, S3 = 3, S4 = 4, S5 = 5, S6 = 6, S7 = 7, S8 = 8;
9
10     reg [7:0] bcd_sequence; // 目标检测序列 (8421 BCD)
11
12     // 初始化目标检测序列为 40 的 BCD 表示
13     always @(posedge clk) begin
14         if (!reset) begin
15             bcd_sequence <= 8'b01000000; // 40 的 BCD 表示
16         end
17     end
18
19     // 状态转移逻辑：逐位检测 BCD 序列
20     always @(c_state, x) begin
21         case (c_state)
22             S0: if (x == bcd_sequence[7]) n_state <= S1; else n_state <= S0; // 检测最高位 (第7位)
23             S1: if (x == bcd_sequence[6]) n_state <= S2; else n_state <= S0;
24             S2: if (x == bcd_sequence[5]) n_state <= S3; else n_state <= S0;
25             S3: if (x == bcd_sequence[4]) n_state <= S4; else n_state <= S0;
26             S4: if (x == bcd_sequence[3]) n_state <= S5; else n_state <= S0;
27             S5: if (x == bcd_sequence[2]) n_state <= S6; else n_state <= S0;
28             S6: if (x == bcd_sequence[1]) n_state <= S7; else n_state <= S0;
29             S7: if (x == bcd_sequence[0]) n_state <= S8; else n_state <= S0; // 检测最低位 (第0位)
30             S8: n_state <= S0; // 检测完成，回到初始状态
31             default: n_state <= S0;
32         endcase
33     end
34
35     // 输出逻辑：当状态到达 S8 时，表示检测完成
36     always @(posedge clk) begin
37         if (c_state == S8)
38             z <= 1'b1; // 完整检测到序列，z = 1
39         else
40             z <= 1'b0; // 未检测完成，z = 0
41     end
42
43     // 状态更新逻辑
44     always @(posedge clk) begin
45         if (!reset)
46             c_state <= S0; // 初始化到 S0 状态
47         else
48             c_state <= n_state; // 状态更新
49     end
50
51 endmodule
52

```

对于数据发生器，在非清零状态下，产生单个编码，用于参加循环。

```

1  module data_generator (clk,clr,dout);
2      input clk,clr;
3      output dout;
4
5      reg [7:0] data;
6      reg dout;
7
8      always@(posedge clk)
9      begin
10         if(!clr)
11         begin
12             dout<=0;
13             data<=8'b10000000;
14         end
15         else
16         begin
17             dout<=data[7];
18             data<={data[6:0],data[7]};
19         end
20     end
21 endmodule

```

如下为顶层文件。

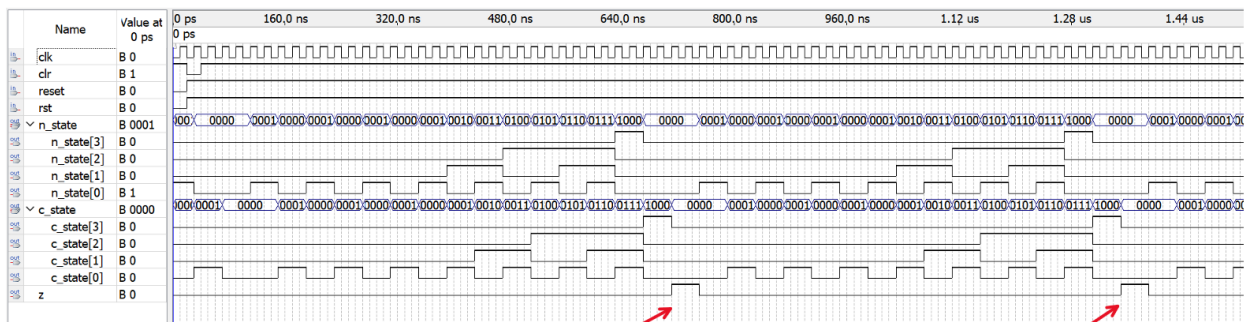
```

1  module top(clk,clr,rst,reset,z,c_state,n_state);
2      input clk,clr,reset,rst;
3
4      output z;
5      output [3:0] c_state,n_state;
6
7      wire clk_lhz;
8      wire dout;
9
10     frequency_divider(.clk_50mhz(clk),.rst(rst),.clk_lhz(clk_lhz));
11     data_generator(.clk(clk_lhz),.clr(clr),.dout(dout));
12     seqdetector(.clk(clk_lhz),.x(dout),.reset(reset),.z(z),.c_state(c_state),.n_state(n_state));
13
14 endmodule

```

4. 仿真验证

如图，在约 8s 运行时间后，出现 z 的短暂真值期，代表检测到了 0100 0000 (40)。



如图中箭头，在程序检测到 40 对应的 BCD 码时，会使 z 为真

5. 引脚分配

如下图，clk 接入内置时钟；clr 与 reset、rst 接入开关以复位；c_state 的 4 个值接入 LED 灯用于展示现态；z 接入 LED 灯以展示程序检测到了 40。

out	c_state[3]	Output	PIN_W15	4
out	c_state[2]	Output	PIN_Y17	4
out	c_state[1]	Output	PIN_R16	4
out	c_state[0]	Output	PIN_T17	5
in	clk	Input	PIN_T1	2
in	clr	Input	PIN_N18	5
out	n_state[3]	Output		
out	n_state[2]	Output		
out	n_state[1]	Output		
out	n_state[0]	Output		
in	reset	Input	PIN_M20	5
in	rst	Input	PIN_AA15	4
out	z	Output	PIN_U12	4

6. 实验现象

初始化过程：将 reset 与 rst 置 1；将 clr 开关先置 0，再置 1，发现 LED 灯每隔 8 秒闪烁一次。reset 开关或 rst 开关置 0 再置 1 后，LED 灯闪烁间隔时间重置。

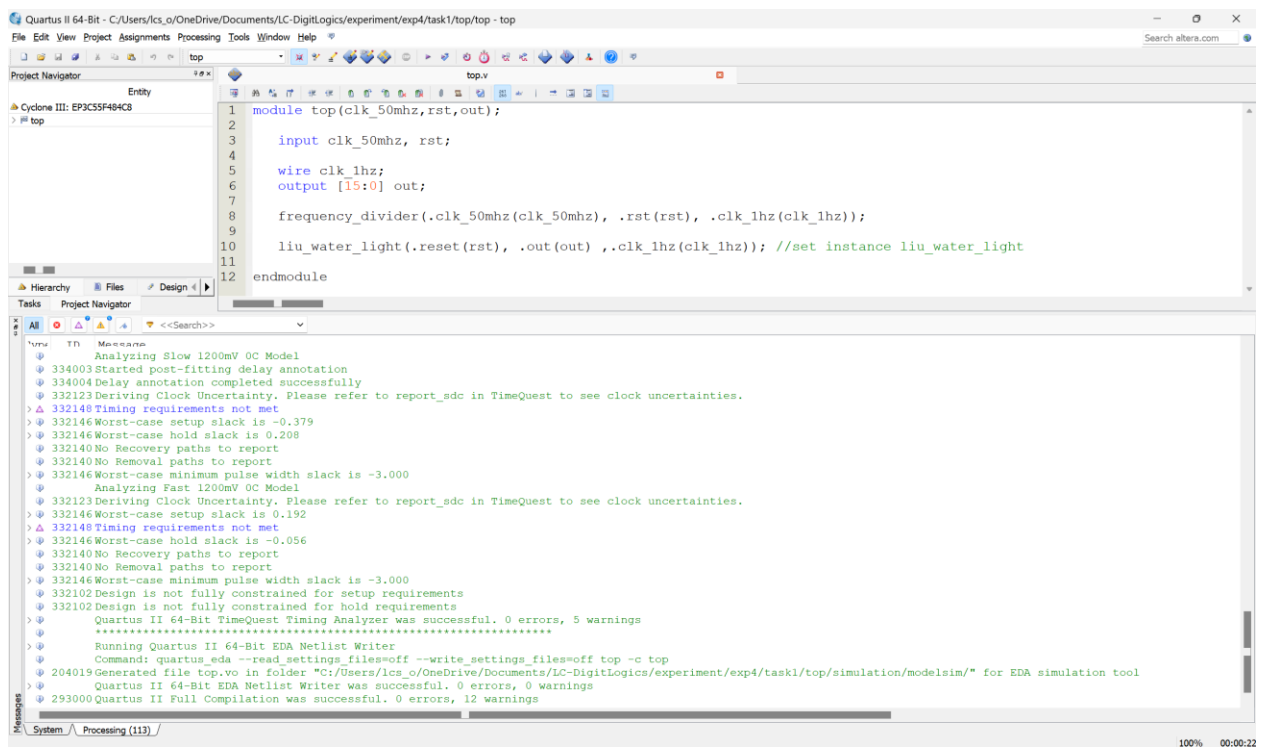
四、总结

对于实验 2，我一开始误将 rst、reset 等变量弄混，导致没有完成初始化，致使 c_state 一直处于 0000 的状态（而非遍历循环状态）。这是因为我对操作不够熟悉。

数字逻辑实验报告

实验 4

附图 1：任务一顶层及综合通过图



附图 2：任务二顶层及综合通过图

